

Person 3 — Risk & Decision Intelligence

Atlas + Compass

Read `00_UNIVERSAL_ARCHITECTURE.md` first. Your scope is intentionally smaller than Person 2's, but Compass is the system that produces the final human-facing recommendation — treat it as high-visibility for the demo even though it's low in system count.

1. What you own

Atlas — external supply-chain / risk intelligence, filtered to *this* procurement's actual exposure (not a generic news feed)

Compass — decision synthesis: takes everyone else's outputs and produces a recommendation with rationale and confidence

2. Tech stack

FastAPI — two services (`atlas/`, `compass/`), or one service with two routers

Supabase (shared project, via `common/supabase_client.py`) for:

cached external signals / event tables (Atlas)

decision/recommendation records (Compass)

reading the Belief Ledger for context Compass needs to reference

LLM calls, routed by stakes via `common/llm_client.py`:

Gemini API (free tier) for Atlas's relevance filtering ("does this external event affect OUR procurement?") and impact estimation

Groq API (free tier) for quick pre-filtering of raw external signals before the heavier relevance call (cuts down how often you invoke the more expensive step)

Claude — reserve this for **Compass's final recommendation + rationale synthesis**. This is the single most human-facing output in the entire system (it's literally what gets shown to the reviewer), so it's worth spending your Claude budget there rather than on Atlas

External data sources for Atlas (pick what's feasible in your timeline — don't try to integrate all of these live):

A free-tier commodity/metals price API, or a free exchange-rate API as a proxy signal

A shipping/port congestion source — free live feeds here are scarce, so default to a curated, clearly-labeled mock dataset unless you find a workable free API in time

News/RSS search (web search) for tariffs, geopolitical events, filtered through the LLM relevance step

If time is short: hardcode 3-5 realistic "external event" rows directly in a Supabase table as seed data and focus your engineering time on the relevance-filtering and impact-mapping logic, which is the actual intellectual contribution of Atlas

3. Capability contracts (register these with Person 1 ASAP)

Atlas

Capability	Input	Output
<code>atlas.assess_external_event</code>	project_id + relevant materials/vendors/routes	list of relevant external events + estimated impact on this project
<code>atlas.check_commodity_exposure</code>	material/commodity list	price trend, exposure level
<code>atlas.check_shipping_risk</code>	shipping route / port refs	congestion/delay risk
<code>atlas.check_geopolitical_risk</code>	vendor/supplier country/region	risk level + relevant events

Output shape:

```
python
class ExternalRisk(BaseModel):
    event_id: str
    event_type: str      # "port_congestion", "tariff", "commodity_price", "geopolitical"
    description: str
    relevance: float     # 0-1 — does this actually touch our procurement?
    estimated_impact: str # human-readable
    affected_entities: list[str] # vendor_id, shipment_id, material, etc.
    confidence: float
```

Design rule: Atlas's core job is the relevance filter. A raw external event feed is not intelligence — `atlas.assess_external_event` should always answer "does this affect OUR procurement" and attach `affected_entities`, not just return generic news.

Compass

Capability	Input	Output
<code>compass.recommend</code>	aggregated context: Sentinel + Trustline + Precedent + Arbiter + Atlas outputs (assembled by ORCHESTRA)	recommendation + rationale + confidence
<code>compass.compare_alternatives</code>	2+ options (e.g. alternate suppliers) + same context	ranked comparison
<code>compass.estimate_impact</code>	proposed decision + project context	cost/schedule/spec/milestone impact

Output shape:

```
python
```

```

class Recommendation(BaseModel):
    recommendation_id: str
    situation_id: str
    recommendation: str
    rationale: str          # human-readable synthesis
    contributing_factors: list[str]  # which specialist findings drove this
    cost_impact: str | None
    schedule_impact: str | None
    risk_exposure: str
    confidence: float
    alternatives_considered: list[str] = []

```

Like Arbiter, Compass should receive its inputs from ORCHESTRA's assembled context (the `context` field of its `AgentTask`), not by calling Sentinel/ Trustline/Precedent/Atlas directly.

4. Build order for you specifically

Phase 0 (with the whole team): freeze `common/` schemas + confirm you're pointed at the shared Supabase project.

Phase 1 — Compass stub, immediately: Build a Compass endpoint that accepts *any* context payload and returns a schema-valid fake recommendation. Give this to Person 1 on day one — it's what lets ORCHESTRA have an end-to-end path before Atlas or Person 2's specialists are fully real. This is more valuable early than a fully-built Atlas.

Phase 2 — Atlas, real but narrow: Pick 1-2 external risk types (e.g. shipping/port + commodity price) and build those end-to-end with real free-tier or realistic-mocked data (stored in a Supabase table), rather than spreading thin across four risk types with none of them convincing.

Phase 3 — Compass, real: Replace the stub with actual synthesis logic once Sentinel/Trustline/Precedent/ Arbiter outputs are available to test against. Route this through Claude (see §2) — this is where most of your prompt-engineering time should go, since the recommendation + rationale quality is what the demo audience will judge most directly.

Phase 4 — Trustline integration point: Make sure `atlas.assess_external_event`'s output can be handed to Person 2's `trustline.update_trust` (via ORCHESTRA) for the "external cause → no reliability penalty" scenario in the universal doc. Coordinate directly with Person 2 on this one handoff — it's the clearest cross-team dependency you have.

5. What you need FROM Person 1 and Person 2

Person 1: finalized `AgentTask`/`AgentResult` schema before building real endpoints; confirmation of how assembled multi-specialist context is passed to Compass.

Person 2: Trustline's `update_trust` input schema, specifically the `external_context` field shape, so Atlas's output slots in cleanly.

Everyone: confirm the shared Supabase URL/keys are distributed before you start writing to shared tables.

6. Demo-readiness checklist for your scope

- ☐ Atlas returns at least one realistic external event with correct `affected_entities` for a seeded scenario
- ☐ Compass produces a recommendation with rationale that visibly draws on 2+ specialist inputs (not just Atlas or just Sentinel alone) — this is what proves the "synthesis" claim

☐ The "port strike → no trust penalty" vs "no external cause → trust penalty" contrast scenario works and is demo-able (coordinate with Person 2)